Universite Abdelmalek Essaadi

Ecole Nationale des Sciences Appliquees de Tanger

Departement Informatique

Examen TP DevOps

Biblio

GitFlow, GitHub Actions et conteneurisation Docker

Filiere : Genie Informatique (GINF2)
Realise par : LACHHAB Mohammed
Encadrants : Pr. Mohammed Ghailani
Pr. Marouane Haddad
Depot : github.com/edwinlabrinth/biblio-devops

Annee universitaire 2026–2027

Table des matières

1 Contexte et objectifs **2**

2 Organisation GitFlow **2**

2.1 Branches et commits 2

2.2 Pull requests et protection 3

3 Integration et livraison continues **4**

4 Conteneurisation Docker **7**

5 Maintenance Git **8**

6 Bilan **9**

Annexe : commandes utiles **10**

1 Contexte et objectifs

Le projet **Biblio** est une application PHP en ligne de commande de gestion de bibliotheque (livres, adherents, emprunts). Ce rapport presente le travail individuel realise autour du code fourni : mise en place d'un depot collaboratif GitFlow, automatisation de l'integration continue avec GitHub Actions, conteneurisation Docker et operations de maintenance Git.

Le travail a ete mene seul. Les roles normalement repartis dans une equipe de trois ont donc ete assures successivement par le meme etudiant. La publication sur Docker Hub n'a pas ete activee : le workflow correspondant existe mais reste conditionne, conformement au perimetre choisi.

Objectif	Resultat
GitFlow	Branches `main`, `develop`, `feature/*`, `release/*`, `hotfix/*`
Commits atomiques	Commits conventionnels, un sujet par commit
Pull requests	9 PR relues puis fusionnees avec un commit de merge
Protection	`main` et `develop` protegees, force push interdit
CI	Lint PHP, 39 tests, quality gate bloquant
Docker	Image multi-stage, tests au build, utilisateur non-root
Release	Tags SemVer `v1.0.0` et `v1.0.1`
Docker Hub	Job prepare mais desactive (hors perimetre)

Table 1 – Synthese des objectifs et de leur etat.

2 Organisation GitFlow

2.1 Branches et commits

Deux branches permanentes structurent le depot : `main` pour les versions publiees et `develop` pour l'integration. Chaque fonctionnalite est developpee dans une branche `feature/*`, puis fusionnee par une pull request avec un commit de merge (pas de fast-forward). La figure 1 montre l'historique obtenu.

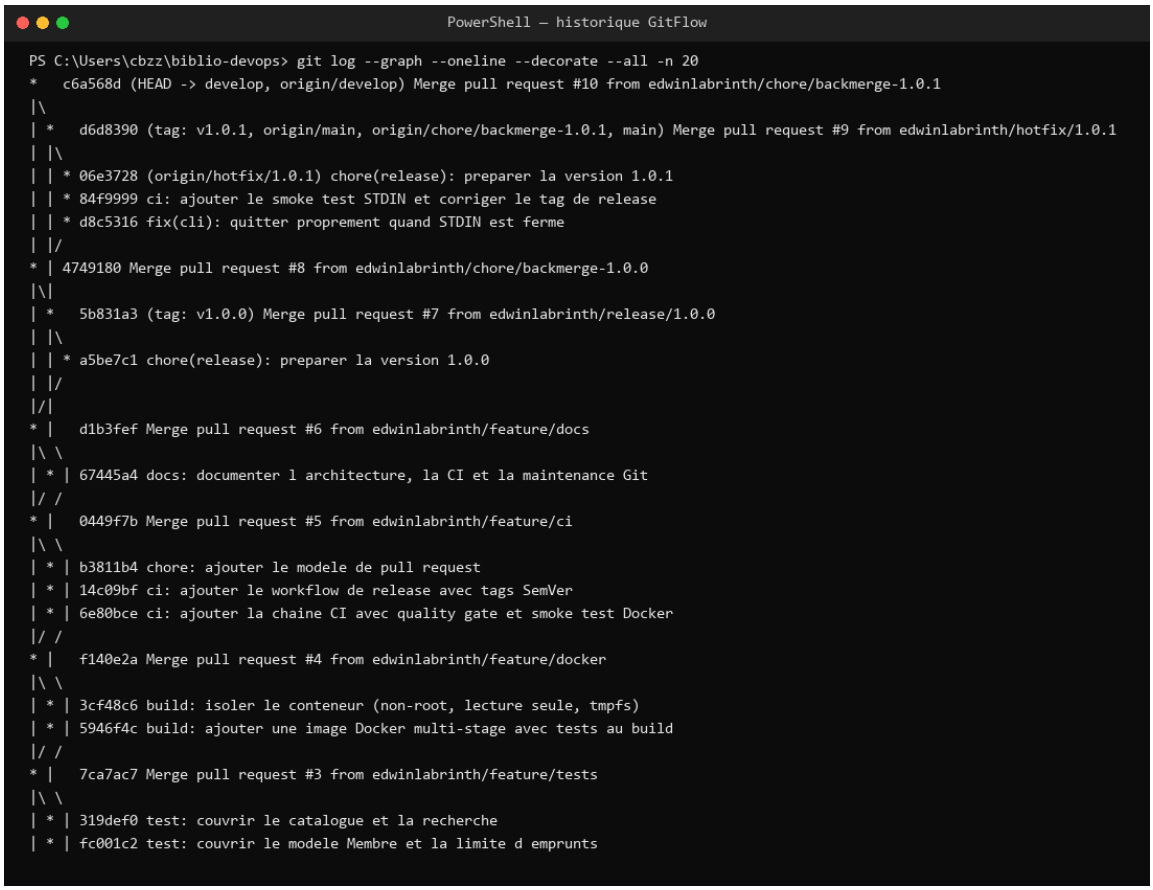


Figure 1 – Historique GitFlow du depot (commande `git log --graph`).

Les commits suivent la convention `type: description` : `feat`, `fix`, `test`, `ci`, `build`, `docs`, `chore`. Le decoupage reste volontairement simple : un commit par modele, un commit par harnais de tests, un commit par brique d'infrastructure.

2.2 Pull requests et protection

Chaque integration passe par une PR avec description, checklist et relecture. Les branches `main` et `develop` sont protegees :

— pull request obligatoire avant fusion ;
— statut `Tests & Quality Gate` obligatoire et a jour ;
— force push et suppression de branche interdits ;
— fusion par commit de merge uniquement (squash et rebase desactives).

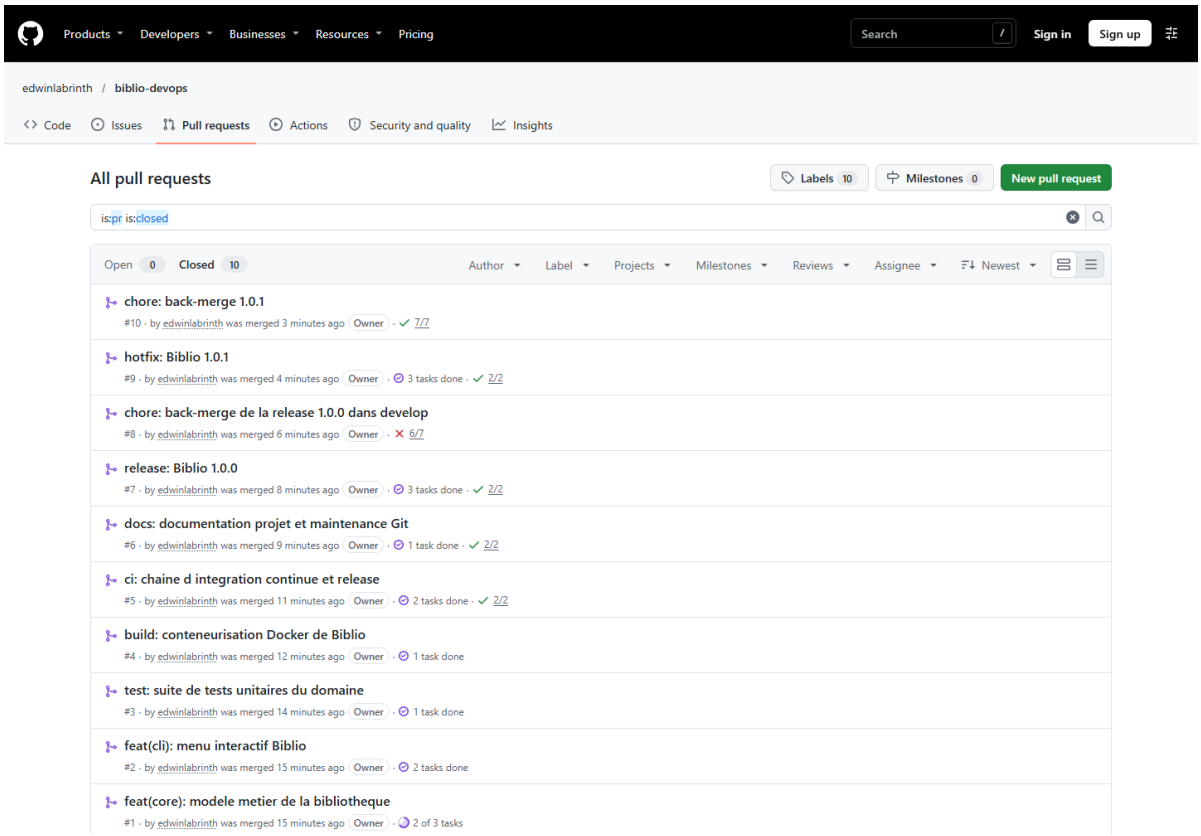


Figure 2 – Pull requests fermees et fusionnees sur GitHub.

Le travail etant individuel, la relecture croisee a ete remplacee par une auto-relecture explicite : un commentaire de revue est depose sur chaque PR avant la fusion, et la CI doit etre verte. Le depot est visible a l'adresse `https://github.com/edwinlabrinth/biblio-devops`.

3 Integration et livraison continues

Le workflow `CI` se declenche sur les push et les pull requests vers `main` et `develop`. Il contient deux jobs principaux :

Job	Etapes
`Tests & Quality Gate`	installation de PHP 8.4, verification de syntaxe, execution des 39 assertions, quality gate bloquante, publication du rapport
`Build & Smoke Test Docker`	construction de l'image (tests au build), verification de l'UID non-root, execution ephemere en lecture seule

Table 2 – Jobs du workflow d'integration continue.

Le quality gate est bloquant : si le script de tests retourne un code non nul, la PR ne peut pas etre fusionnee. La figure 3 montre les executions vertes sur la branche `develop`.

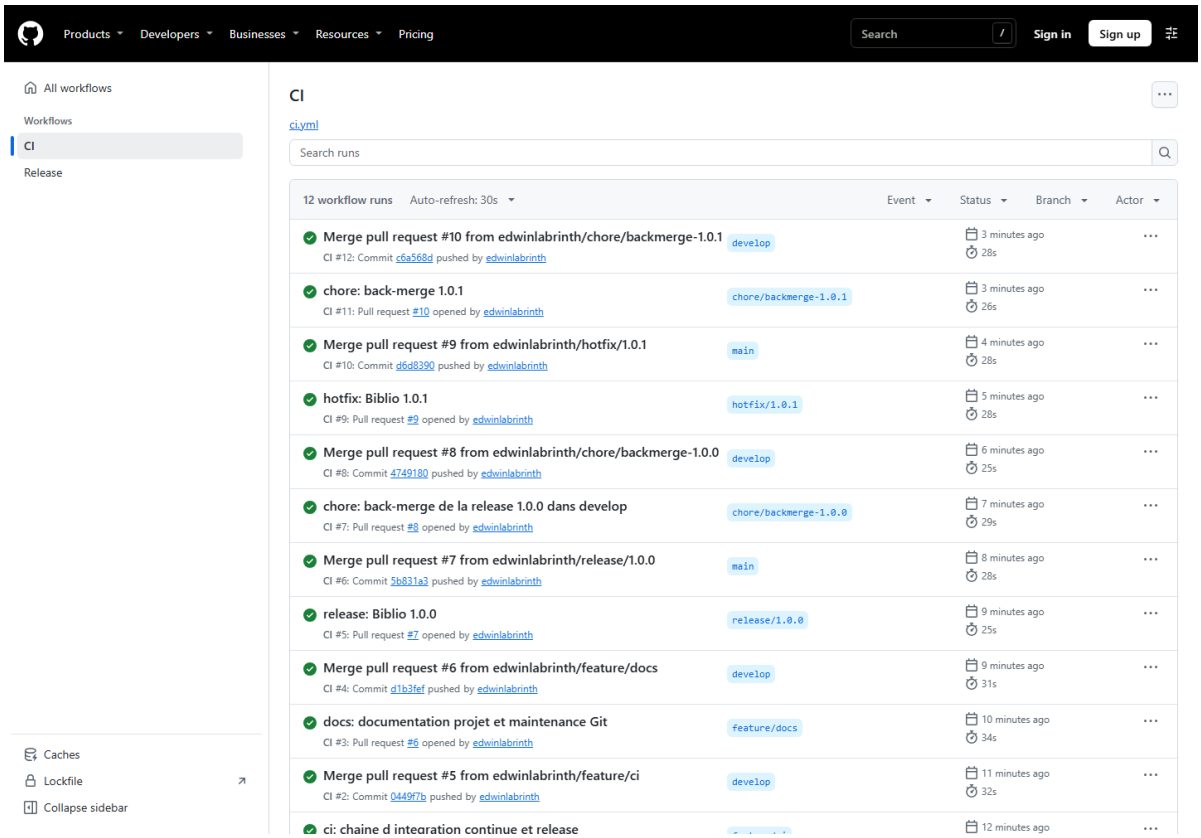


Figure 3 – Executions du workflow CI sur GitHub Actions.

Le workflow `Release` est declenche par un tag `vX.Y.Z`. Il rejoue les tests, construit l'image et calcule automatiquement les tags SemVer `X.Y.Z`, `X.Y`, `X` et `latest`. La figure 4 montre l'execution reussie de la version 1.0.1.

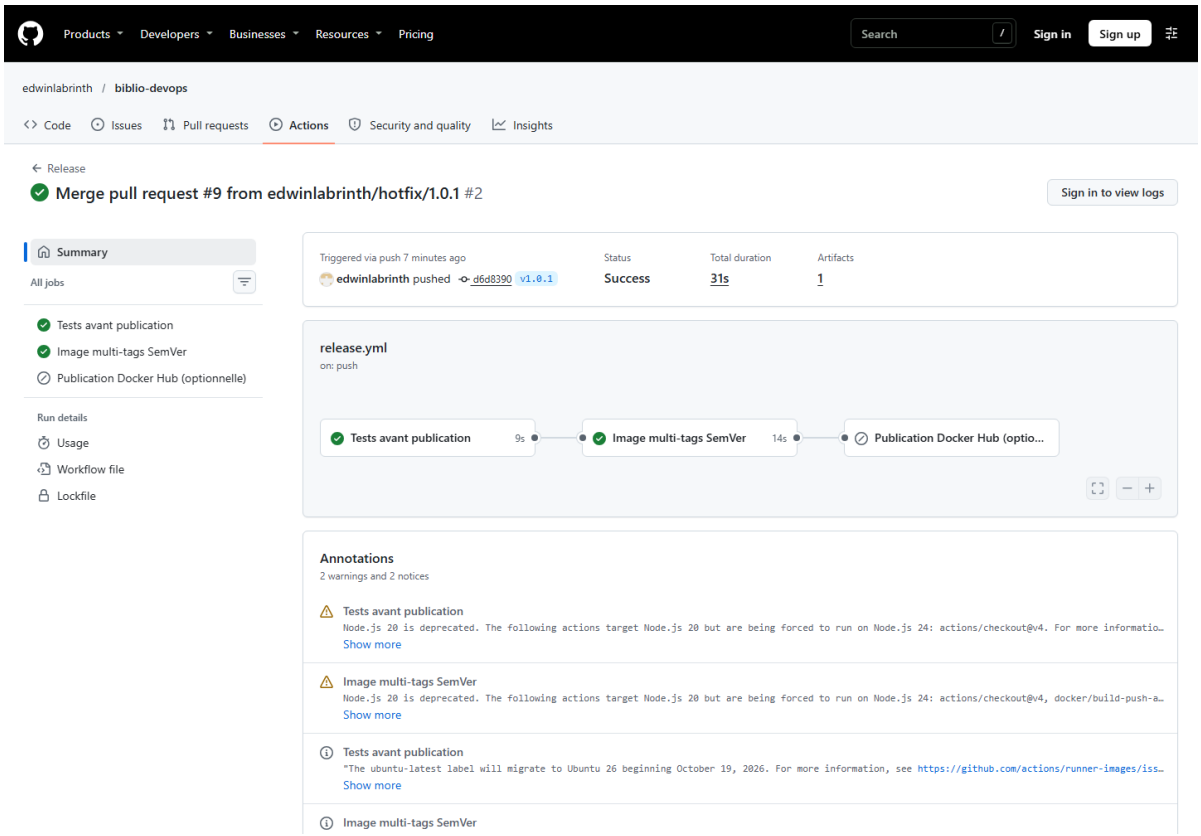


Figure 4 – Workflow de release declenche par le tag `v1.0.1`.

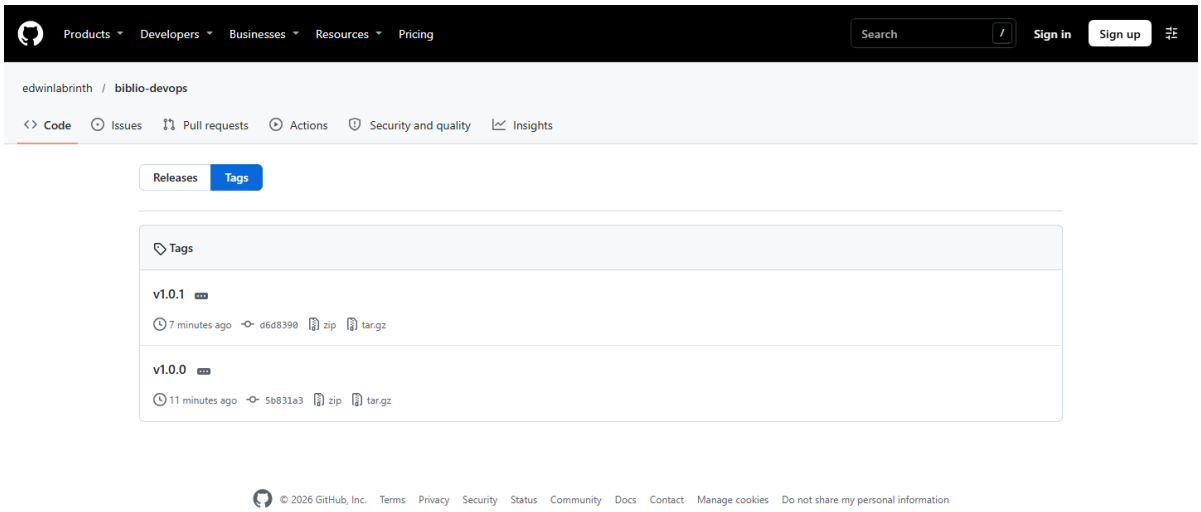


Figure 5 – Tags SemVer publies sur le depot.

4 Conteneurisation Docker

L'image est construite en trois etapes : `base`, `test` et `runtime`. L'etape `test` execute `php tests/run.php` pendant le build ; l'image finale herite de cette etape, donc elle ne peut pas etre produite si un test echoue. Le conteneur s'execute avec un utilisateur non privilegie et peut tourner en lecture seule.

```
FROM php:8.4-cli-alpine AS base
WORKDIR /app
COPY autoload.php index.php VERSION ./
COPY src ./src
COPY tests ./tests
RUN addgroup -S biblio && adduser -S -G biblio biblio
USER biblio

FROM base AS test
RUN php tests/run.php

FROM test AS runtime
ENTRYPOINT ["php", "index.php"]
```

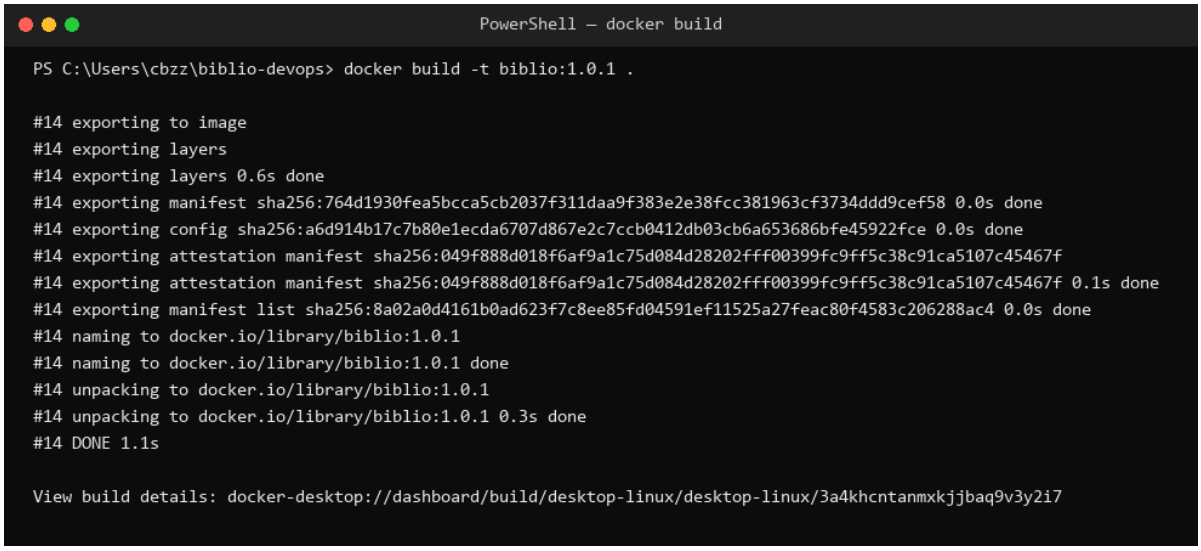


Figure 6 – Construction de l'image : les tests sont executes au build.

L'execution est ephemere et isolee :

```
docker run --rm -i --read-only --tmpfs /tmp \
  --security-opt no-new-privileges:true biblio:1.0.1
```

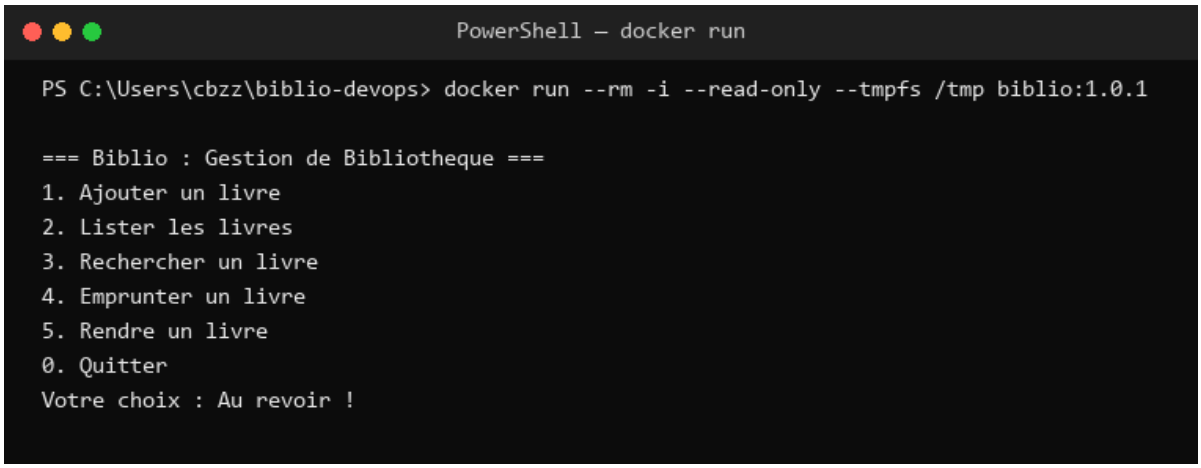


Figure 7 – Execution isolee du conteneur avec une entree utilisateur.

Le fichier `docker-compose.yml` reprend les memes options (`read_only`, `tmpfs`, `no-new-privileges`) pour un usage local. Les tests locaux restent executables sans Docker avec `php tests/run.php` (figure 8).

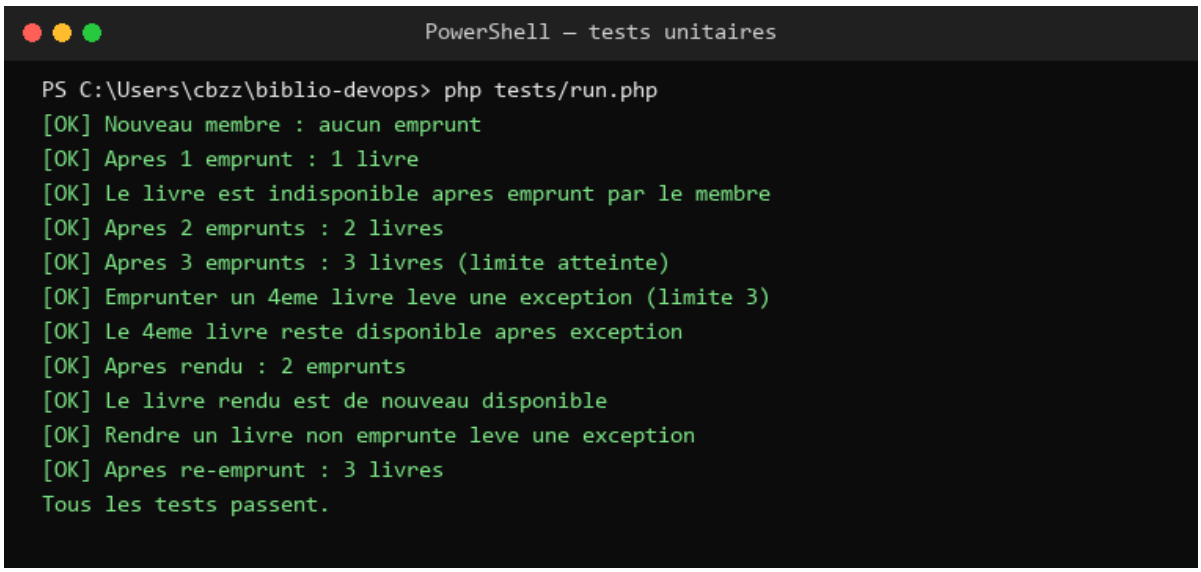


Figure 8 – Suite de tests : 39 assertions, aucune erreur.

5 Maintenance Git

Les operations de maintenance ont ete documentees dans `docs/maintenance.md`. L'inspection de l'historique et le reflog ont ete utilises pour retrouver les commits et verifier l'etat des branches (figure 9).

PowerShell — inspection et maintenance

```
PS C:\Users\cbzz\biblio-devops> git blame -L 60,70 src/Bibliotheque.php
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 60)     public function rechercher(string $mot): array
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 61)     {
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 62)         $resultats = [];
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 63)         foreach ($this->livres as $livre) {
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 64)             if (stripos($livre->getTitre(), $mot) !== false
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 65)                 || stripos($livre->getAuteur(), $mot) !== false) {
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 66)                 $resultats[] = $livre;
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 67)             }
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 68)         }
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 69)         return $resultats;
e7c3ce11 (LACHHAB Mohammed 2026-10-30 11:53:49 +0100 70)     }

PS C:\Users\cbzz\biblio-devops> git branch -a
* develop
  main
  remotes/origin/chore/backmerge-1.0.1
  remotes/origin/develop
  remotes/origin/hotfix/1.0.1
  remotes/origin/main

PS C:\Users\cbzz\biblio-devops> git reflog -6
c6a568d HEAD@{0}: checkout: moving from develop to develop
c6a568d HEAD@{1}: pull --ff-only origin develop: Fast-forward
4749180 HEAD@{2}: checkout: moving from chore/backmerge-1.0.1 to develop
d6d8390 HEAD@{3}: checkout: moving from main to chore/backmerge-1.0.1
d6d8390 HEAD@{4}: checkout: moving from main to main
d6d8390 HEAD@{5}: pull --ff-only origin main: Fast-forward
```

Figure 9 – Inspection de l'historique, attribution des lignes et reflog.

Besoin	Commande / principe
Inspection	`git log --graph`, `git show`, `git blame`
Recherche de regression	`git bisect start / good / bad / reset`
Remise a zero controlee	`git reset --soft`, `--mixed`, recuperation par `reflog`
Compensation	`git revert <commit>`, sans reecrire l'historique

Table 3 – Commandes de maintenance prevues et documentees.

La recherche dichotomique n'a pas ete executee sur une regression reelle du domaine : la suite de tests et la CI ont detecte les erreurs au moment des PR. La procedure reste documentee et applicable telle quelle. Le hotfix 1.0.1 illustre la compensation d'un defaut decouvert apres la release 1.0.0 : sortie infinie du menu lorsque l'entree standard est fermee.

6 Bilan

- Depot GitFlow complet, branches protegees et historique lisible.
- 9 pull requests relues, CI verte avant chaque fusion.
- 39 assertions, quality gate bloquant et rapport de tests archive.
- Image Docker multi-stage, non-root, testee au build et executee en mode ephemere.
- Deux versions SemVer publiees : `v1.0.0` et `v1.0.1`.
- Publication Docker Hub volontairement non activee.

Le principal point d'adaptation est le travail individuel : la revue croisee et la repartition des roles ont ete remplacees par une auto-relecture et une execution sequentielle des taches. Le resultat reste tracable dans les PR et dans l'historique Git.

Annexe : commandes utiles

```
# tests
php tests/run.php

# image et execution
docker build -t biblio:1.0.1 .
docker run --rm -it --read-only --tmpfs /tmp biblio:1.0.1

# etat du depot
git log --graph --oneline --decorate --all
git tag -n
git status -sb
```